

sketch_may7a.ino

```

1  #include <Arduino.h>
2  #include <BLEDevice.h>
3  #include <BLEUtils.h>
4  #include <BLEScan.h>
5  #include <BLEAdvertisedDevice.h>
6  BLEScan* pBLEScan;
7  BLEClient* pClient = nullptr; // Global variable to hold the BLEClient object int speakerpin=15; // speaker is connected to pin 15
8  class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {
9  void onResult(BLEAdvertisedDevice advertisedDevice)
10 {
11   Serial.print("Found BLE Device: ");
12   Serial.println(advertisedDevice.getName().c_str());
13   Serial.print("MAC Address: ");
14   String s=advertisedDevice.getAddress().toString().c_str();
15   Serial.println(s);
16   if (s=="XX:XX:XX:XX:XX:XX" || s=="XX:XX:XX:XX:XX:XX")//desired MAC address {
17     digitalWrite(speakerpin, HIGH);
18     delay(5000);
19     digitalWrite(speakerpin, LOW);
20     delay(1800000); // buffer time of half hour
21     delay(1800000); // additional buffer time of half hour
22   }
23 }
24 };
25 void setup()
26 {
27   Serial.begin(115200);
28   Serial.println("Scanning for BLE devices...");
29   pinMode(speakerpin, OUTPUT);
30   BLEDevice::init("");
31   pBLEScan = BLEDevice::getScan();
32   pBLEScan->setAdvertisedDeviceCallbacks(new MyAdvertisedDeviceCallbacks()); pBLEScan->setActiveScan(true);
33   pBLEScan->start(10); // Scan for 10 seconds
34 }
35 void loop()
36 {
37   BLEScanResults foundDevices = pBLEScan->start(5, false);// scans for BLE devices pBLEScan->clearResults(); // delete results fromBLE
38 }
39

```

Fig. 5: Snippet of the source code

A MAC address is a 12-digit hexadecimal number assigned to each device connected to the network. Primarily specified as a unique identifier during device manufacturing, the MAC address is often found on a device's network interface card (NIC).

A MAC address is represented by 12 hexadecimal digits (0-9 and A-F) separated by colons or dashes. For instance, 01:0A:95:9D:67:16 or 01-0A-95-9D-67-16. Each hexadecimal digit represents four bits; thus, every pair of digits signifies one byte or octet. The initial six digits in a MAC address indicate the NIC manufacturer, known as organisationally unique identifier (OUI), while the remaining six digits are assigned by

the manufacturer to identify the NIC.

To obtain the MAC address of your smartwatch, on the home screen of your watch, press the down button and go to Settings. The device name is your watch's Bluetooth name. The 12 alphanumeric characters under MAC are your smartwatch's MAC address.

Once the relevant MAC address is obtained, a code can be built to scan all neighbouring BLE-enabled devices within the range and identify specific ones. In the proposed setup, on detecting the desired MAC address, a musical tone is activated using a transistor (BC547) and a speaker assembly.

To ensure that the speaker does not play the tone continuously, its

duration is limited to 5 seconds. Further, even if the identified MAC address remains within the range of detection, the same will continue to be detected by the code, but it will not activate the speaker in any way. As a precautionary measure to avoid detection, an additional (buffer) timer of 3,600,000 ms = an hour is set up to wind up everything before the tone starts playing again.

Circuit and working

Fig. 3 shows the circuit diagram of the BLE-based nearby person notifier. It is built around the ESP32 board (MOD1), smartwatch, BT66 (IC1), NPN transistor BC547 (IC2), a small speaker, and a few other components.